Project Report (SDA)

Title:

Karachi Smart Transit System (KSTS)

Submitted by:

Ghulam Mujtaba Qureshi 24K-0535

Adil Saeed 24K-0705

Haseem Samo 24K-0666

Zaid Faraz 24K- 0636

Course: CS3004 Software Design and Analysis
Instructor: Mr. Jibran Rasheed Khan
Date: April 30, 2026

Chapter 1: Introduction

System Overview

The Karachi Smart Transit System (KSTS) is a centralized, database-driven platform designed to improve the management and efficiency of public transportation in Karachi. The system integrates passenger services, administrative controls, and real-time operational features into a single unified solution.

Passengers can register, book trips, make payments, and receive digital tickets. Administrators manage routes, buses, drivers, and schedules while generating analytical reports. The system enhances operational efficiency, reduces scheduling conflicts, and supports data-driven decision-making.

Problem Statement

Karachi's transport system suffers from:

- Lack of centralized management
- Inefficient scheduling
- Poor seat availability tracking
- Manual and error-prone booking/payment processes

KSTS addresses these issues by providing a real-time, integrated platform that improves both passenger experience and administrative control.

Scope

The system includes:

- User registration and authentication
- Trip booking and cancellation
- Seat management and ticket generation
- Online and recharge card payments
- Route, bus, and driver management
- Trip scheduling
- Reporting and analytics

Out of Scope:

- Voice-based booking (future enhancement)

Objectives

- Enable efficient passenger booking and ticketing
- Provide powerful admin management tools
- Ensure secure payment processing
- Support data-driven decisions
- Maintain scalability and reliability

Users and Stakeholders

Primary Users:

- Passenger
- Admin
- Driver

Stakeholders:

- Transport Authorities
- Payment Gateway Providers

System Requirements Summary

Functional Requirements:

- User authentication
- Trip booking/cancellation
- Payment processing
- Resource management (routes, buses, drivers)
- Trip scheduling
- Reporting

Non-Functional Requirements:

- Reliability
- Scalability
- Security
- Usability

Software Process Model

The project follows an **Agile/Iterative Model**, allowing:

- Incremental development
- Continuous testing
- Flexibility in changing requirements

Chapter 2: System Analysis and Modeling

The system is modeled using UML diagrams including use case, class, activity, sequence, deployment, and timing diagrams. Additionally, the architecture follows the **4+1 model** to represent different perspectives.

Architectural Overview (4+1 Model)

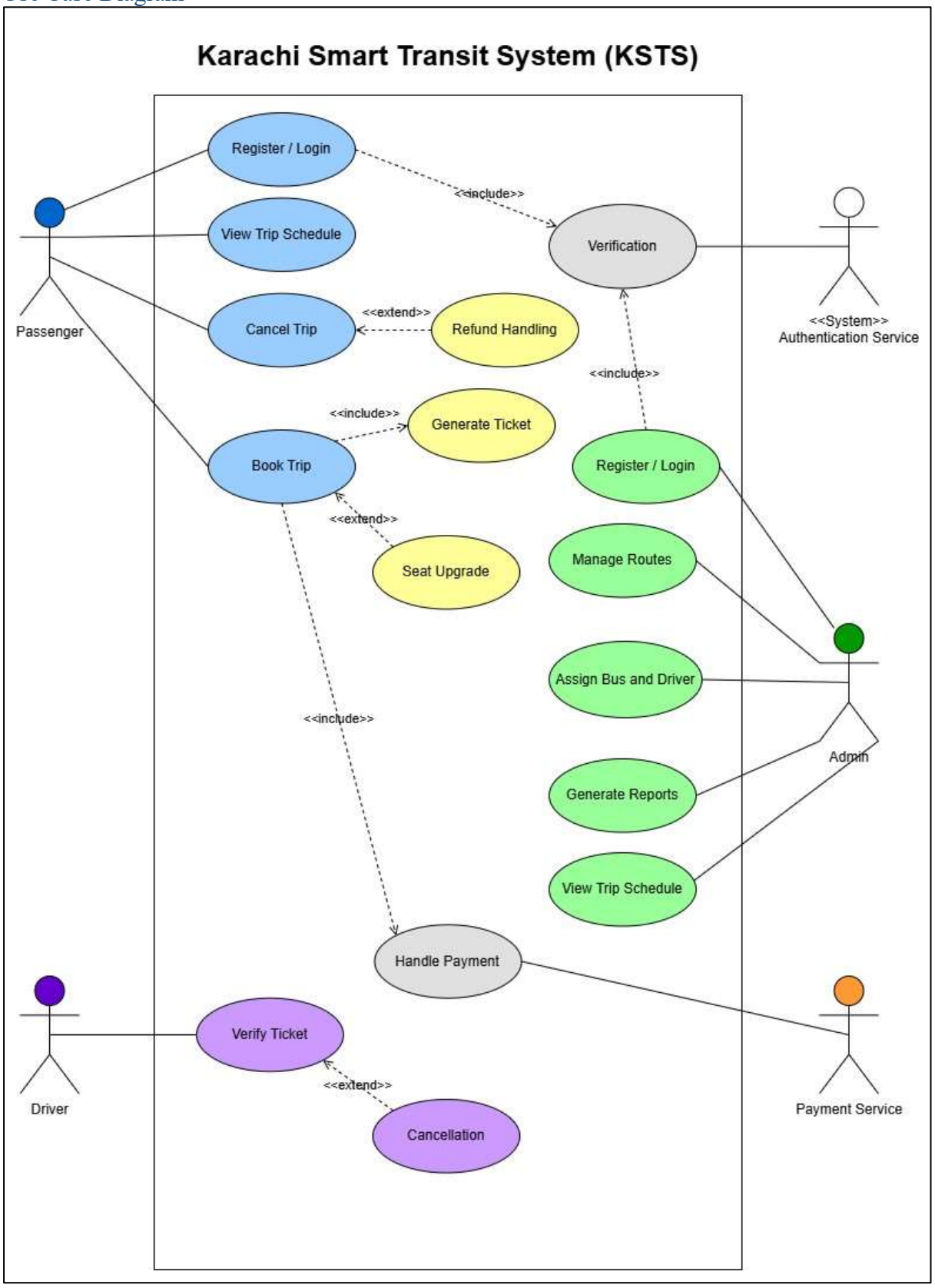
- **Logical View:** Defines system structure (classes and relationships)
- **Process View:** Shows runtime behavior and concurrency
- **Development View:** Describes system modules and code organization
- **Physical View:** Represents deployment architecture
- **+1 Use Case View:** Captures user interactions

Conclusion

The Karachi Smart Transit System provides a scalable and efficient solution for modern public transport management. It improves service quality, enhances user experience, and enables better decision-making through centralized data and automation.

Chapter 2: System Analysis and Modeling

Use Case Diagram



Use Case Description Tables

Register/Login

Use Case Name	Register/Login
Use Case Description	Allows users (Passenger, Admin) to create an account or log into the system.
Actors	Passenger (Primary), Admin (Other)
Stakeholders	System Administrator, Registered Users
Description	Provides secure access to system functionalities.
Includes	None
Extends	None
Input	Username, Password, or Registration Details
Pre-conditions	User must have access to the system interface.
Flow of events	1. User opens login/registration page. 2. Enters credentials or registration details. 3. System verifies and grants access.
Alternate	Invalid credentials or multiple failed attempts → account locked.
Post-conditions	User session created and access granted.

Book Trip

Use Case Name	Book Trip
Use Case Description	Allows passengers to select and reserve a trip.
Actors	Passenger (Primary), Payment Service (Other)
Stakeholders	Passengers, Admin
Description	Enables users to plan and confirm travel.
Includes	Handle Payment, Generate Ticket
Extends	Seat Upgrade
Input	Trip details (origin, destination, date, payment info)
Pre-conditions	Passenger must be logged in.
Flow of events	1. Passenger selects trip and timing. 2. System checks seat availability. 3. Passenger confirms and pays. 4. Payment processed. 5. Ticket generated.
Alternate	Payment failure or no available seats → booking canceled.
Post-conditions	Trip booked successfully; ticket stored in passenger account.

Generate Ticket

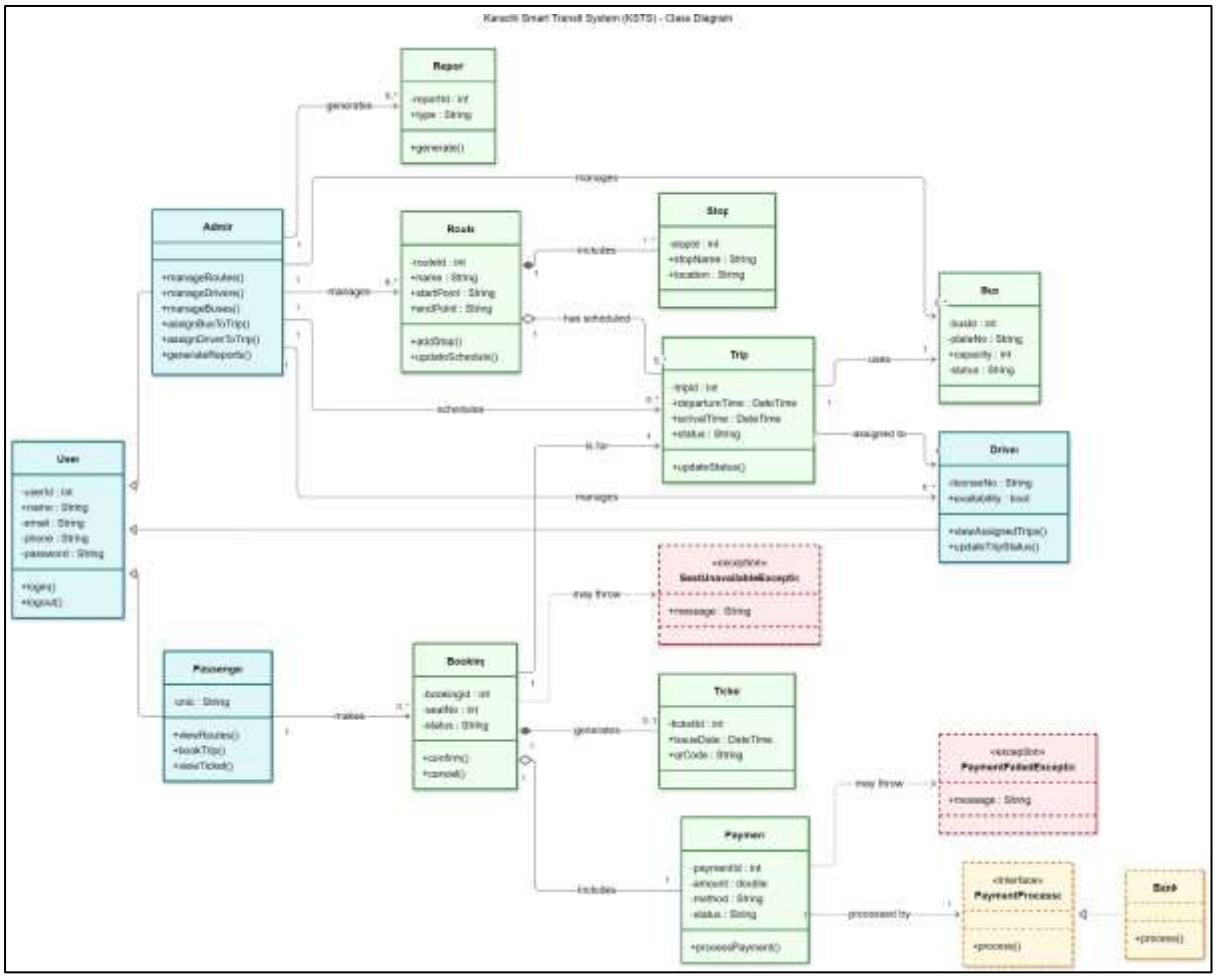
Use Case Name	Generate Ticket
Use Case Description	Produces a digital or printable ticket after booking.
Actors	Passenger (Primary), Admin (Other)
Stakeholders	Passenger, Transit Authority
Description	Confirms passenger's booking and travel authorization.
Includes	None
Extends	None
Input	Booking ID, Payment Confirmation
Pre-conditions	Payment must be successful.
Flow of events	1. System generates ticket details. 2. Ticket assigned to user. 3. Notification sent.
Alternate	Ticket generation error → user notified.
Post-conditions	Ticket generated and available for verification.

Assign Bus and Driver

Use Case Name	Assign Bus and Driver
Use Case Description	Allows admin to assign drivers and buses to routes.
Actors	Admin
Stakeholders	Drivers, Passengers
Description	Ensures each scheduled trip has an assigned driver and vehicle.
Includes	None
Extends	None
Input	Driver ID, Bus ID, Route ID
Pre-conditions	R
Flow of events	1. Admin selects route. 2. Assigns driver and bus. 3. System updates and notifies driver.
Alternate	Driver unavailable → admin selects another.
Post-conditions	Bus and driver successfully assigned.

Class Diagram

a. Classes, Attributes, Methods, Relationships



b. Boundary, Control, Entity Classes (BCE Model)

Boundary

Boundary classes represent user interfaces or interaction points between the system and external actors.

- **PassengerUI:** Interface for passengers to browse routes, book trips, and view tickets.
- **DriverUI:** Interface for drivers to view assigned trips and update trip status.
- **AdminUI:** Interface for administrators to manage routes, buses, trips, and reports.

Control

Control classes manage the flow of logic for specific use cases.

- **BookingController:** Coordinates booking creation, seat assignment, ticket generation, and payment handling.
- **PaymentController:** Manages payment initialization, validation, and selection of processor.
- **TripController:** Handles trip scheduling, driver assignment, and bus assignment.
- **RouteController:** Manages route creation, stop management, and schedule updates.
- **ReportController:** Generates reports requested by the Admin.

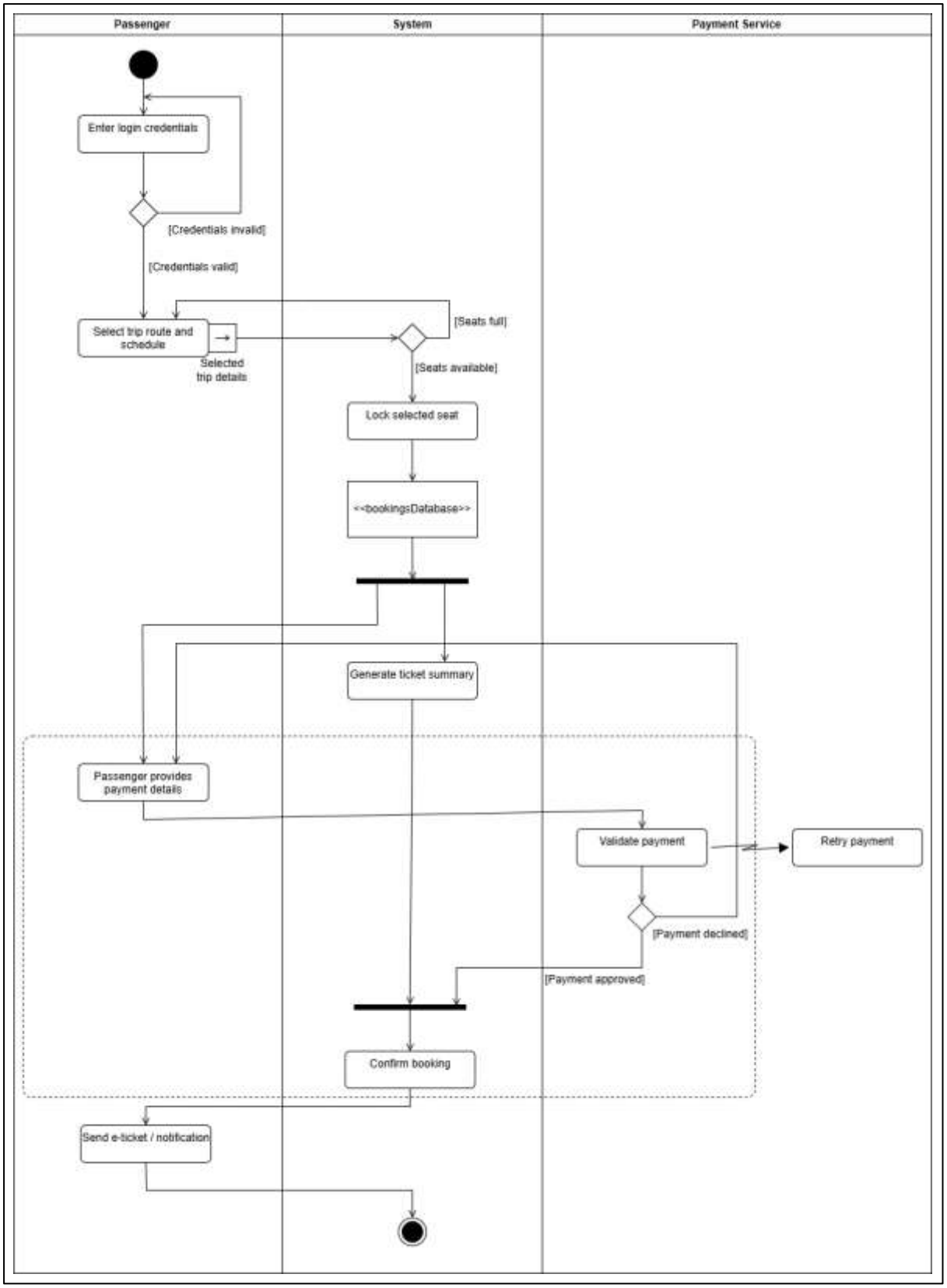
Entity

Entity classes represent persistent domain concepts stored in the system.

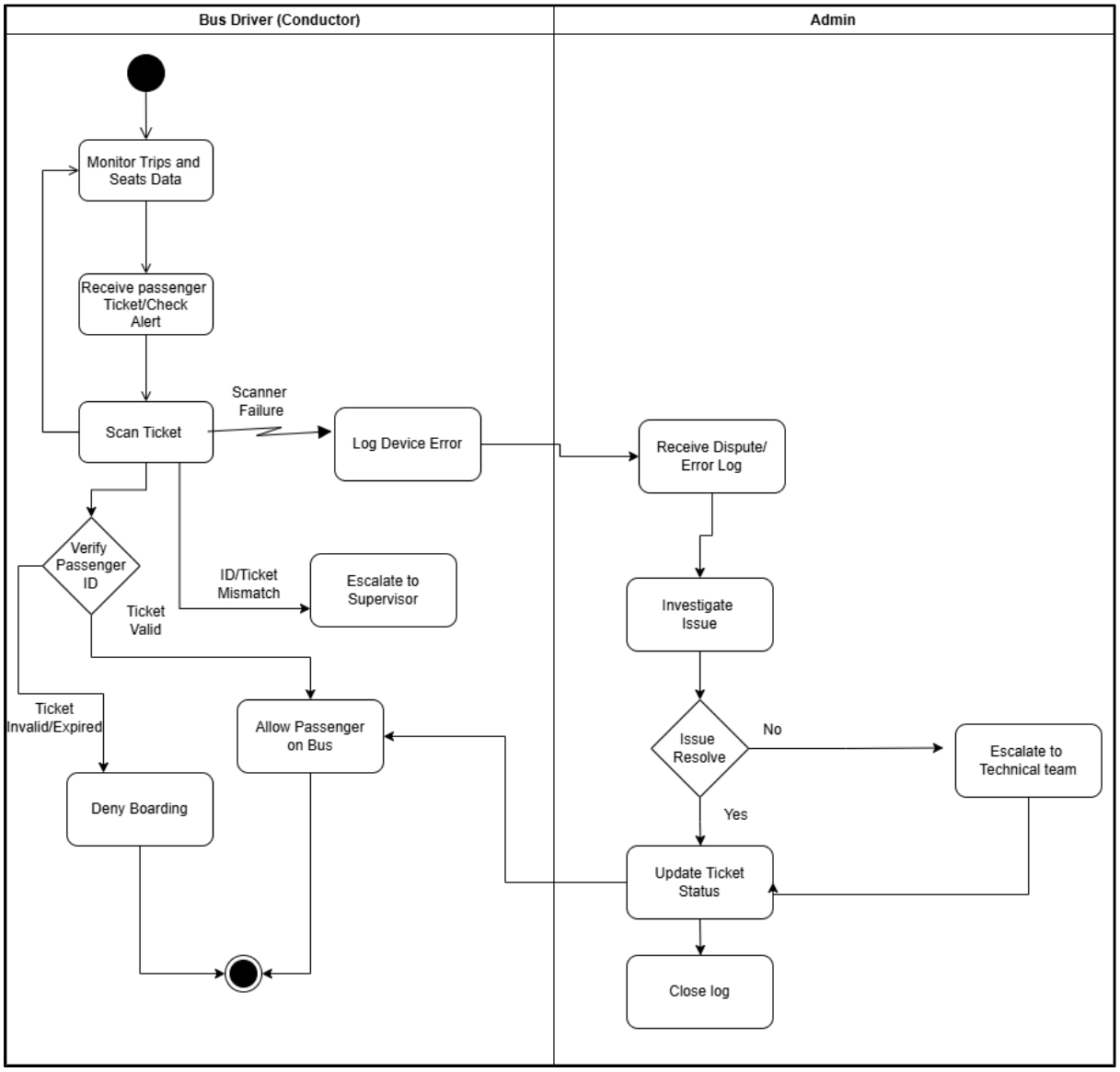
- User
- Passenger
- Driver
- Admin
- Route
- Stop
- Bus
- Trip
- Booking
- Ticket
- Payment
- Report
- PaymentProcessor (interface)
- Bank (implements PaymentProcessor)

Activity Diagrams

Passenger Book Trip



Driver Ticket Validate



Activity Diagram - Passenger Book Trip Description

This Activity Diagram illustrates the step-by-step workflow involved in the passenger trip booking process. It begins with the passenger logging into the system and selecting trip details such as origin, destination, and travel time.

The system then checks seat availability and proceeds to the payment stage if seats are available. Upon successful payment, the system generates a ticket and confirms the booking. Decision nodes are used to handle alternate scenarios such as seat unavailability or payment failure, ensuring robustness in the process flow.

This diagram is useful for visualizing business logic, decision-making points, and the overall flow of activities from start to end.

Activity Diagram - Driver Ticket Validate Description

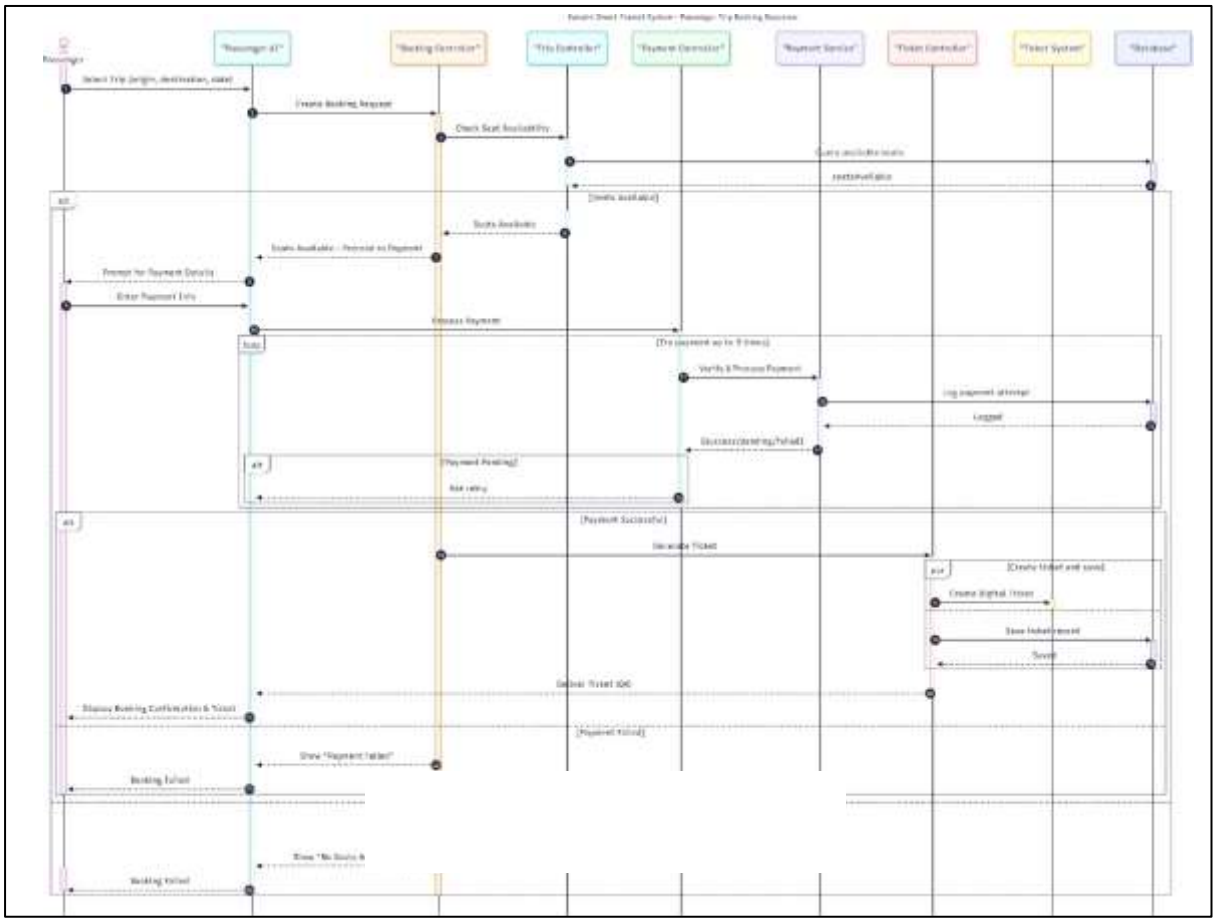
This Activity Diagram illustrates the step-by-step workflow involved in the passenger trip booking process. It begins with the passenger logging into the system and selecting trip details such as origin, destination, and travel time.

The system then checks seat availability and proceeds to the payment stage if seats are available. Upon successful payment, the system generates a ticket and confirms the booking. Decision nodes are used to handle alternate scenarios such as seat unavailability or payment failure, ensuring robustness in the process flow.

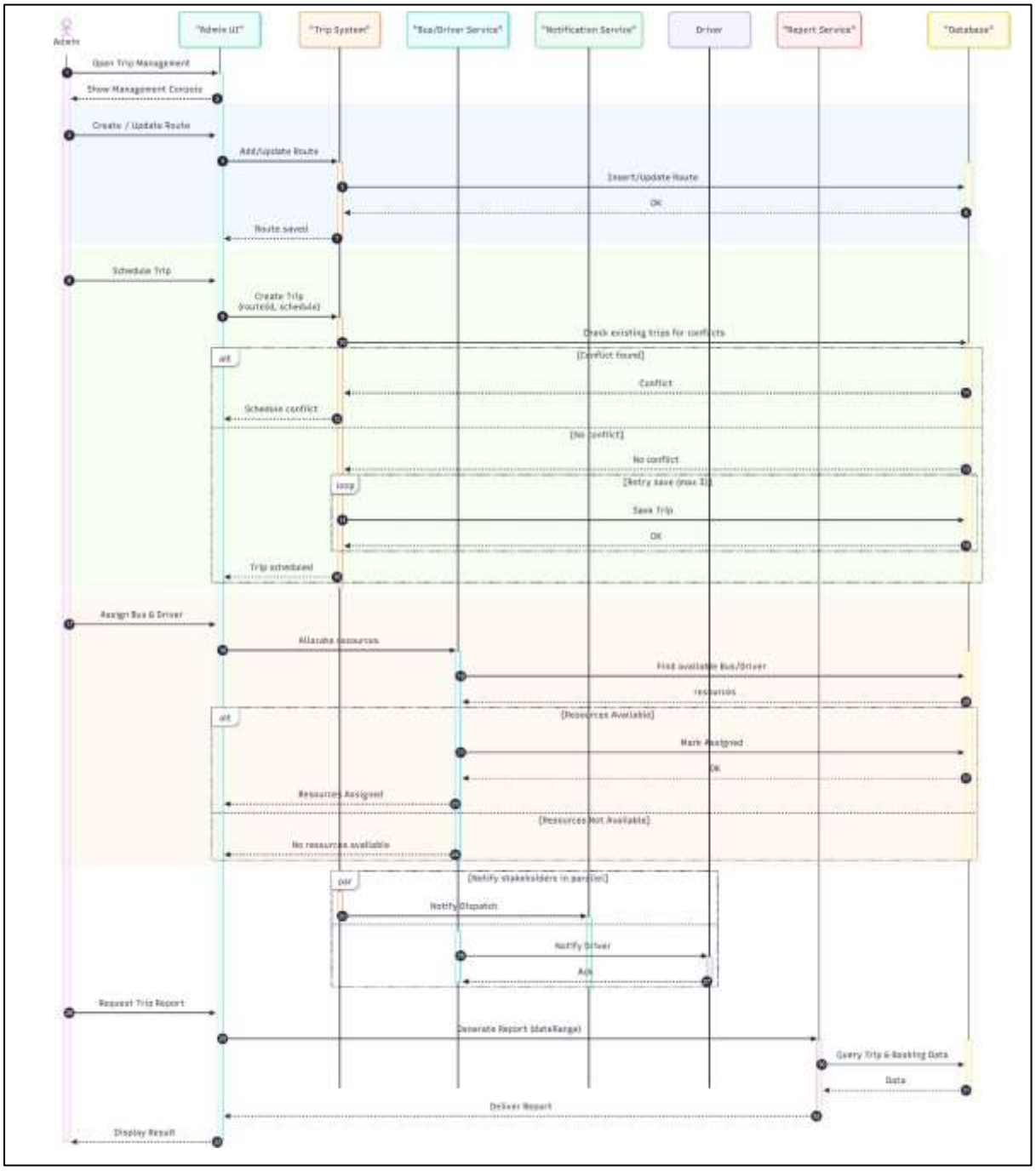
This diagram is useful for visualizing business logic, decision-making points, and the overall flow of activities from start to end.

Sequence Diagrams

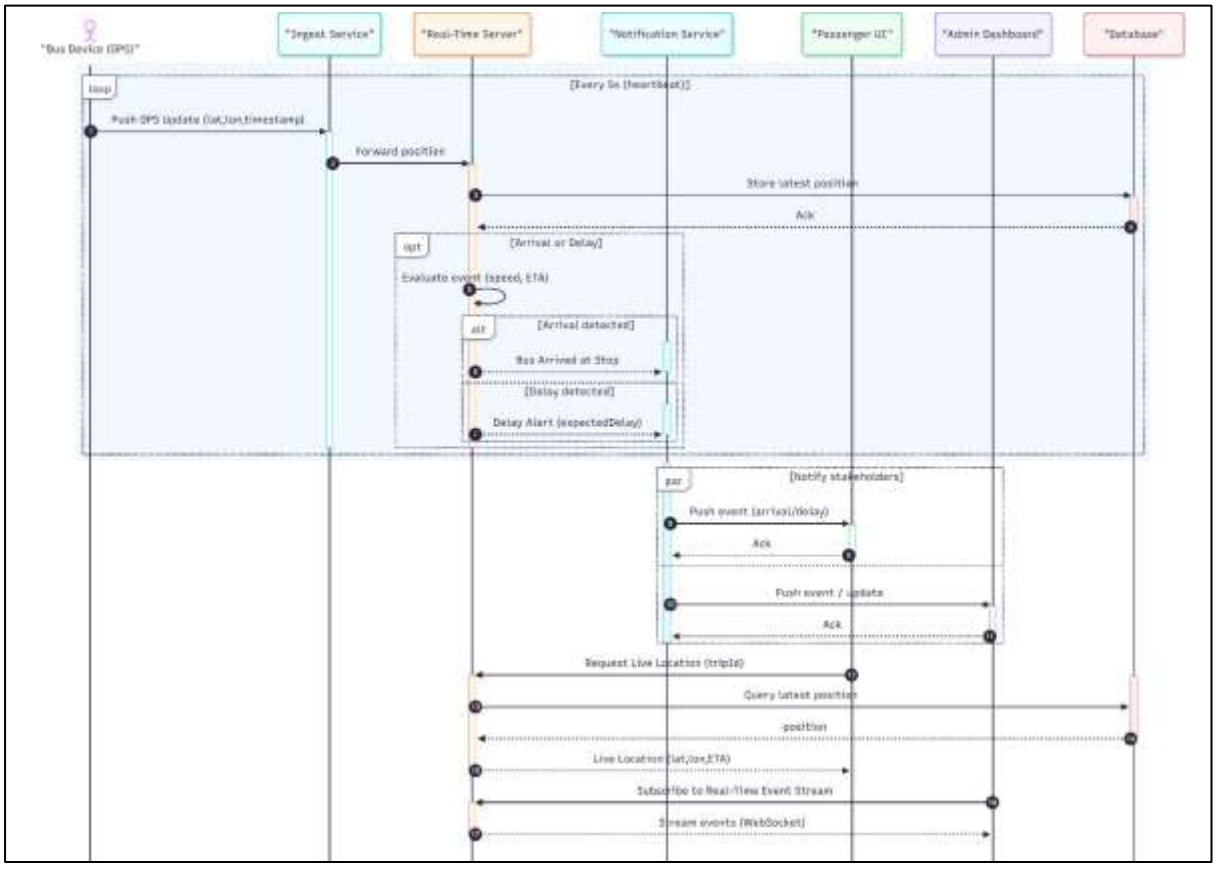
Passenger Trip Booking



Admin Trip Management



Real-Time Bus Tracking - Notification Flow



Sequence Diagram – Passenger Trip Booking

The Sequence Diagram for Passenger Trip Booking illustrates the dynamic interaction between different system components over time. It shows how the Passenger interacts with the user interface, which communicates with backend controllers such as BookingController and PaymentController.

The process includes trip selection, seat availability check, payment processing through the Payment Service, and ticket generation. Messages are exchanged in a sequential order, highlighting the flow of control and data between objects.

This diagram helps in understanding how different components collaborate to complete the booking process and ensures proper system coordination.

Sequence Diagram – Admin Trip Management

This diagram represents the interaction between the Admin and system components for managing trips. The admin initiates actions such as creating routes, assigning buses and drivers, and scheduling trips.

The system processes these requests through controllers and updates the database accordingly. It may also notify drivers about their assigned trips. The sequence of messages demonstrates how administrative operations are handled efficiently.

This diagram is essential for understanding backend workflows and administrative control mechanisms.

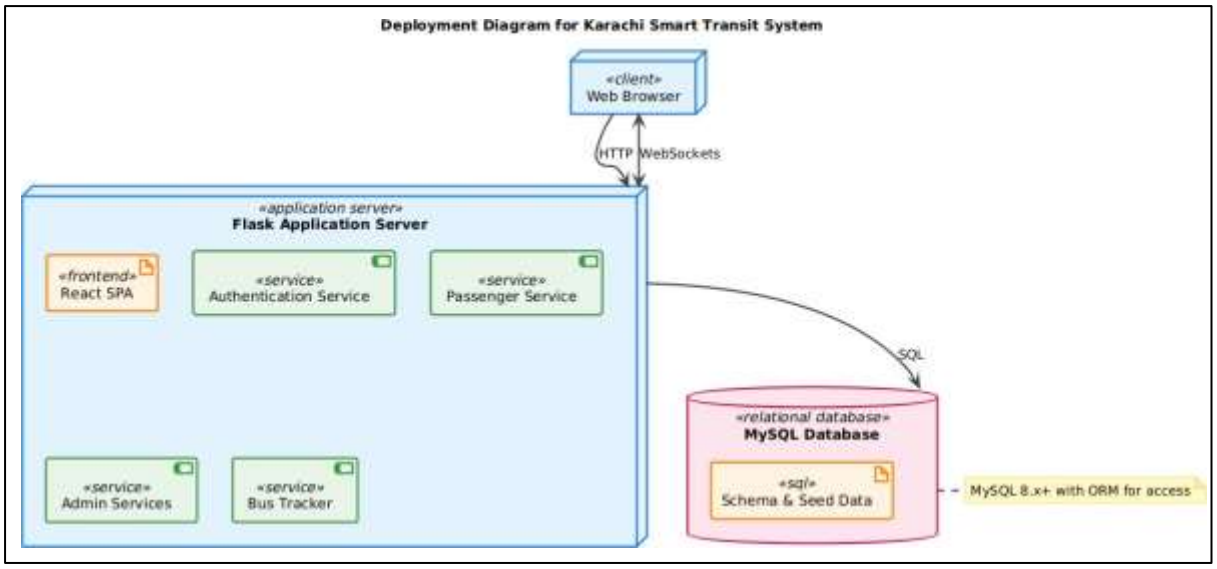
Sequence Diagram – Real-Time Bus Tracking

This diagram illustrates the real-time communication flow for tracking buses. GPS devices installed in buses continuously send location updates to the system.

The system processes this data and updates the database, after which notifications are sent to passengers and admins using WebSocket connections. This ensures that users receive live updates without refreshing the application.

The diagram highlights asynchronous communication and real-time data handling, which are critical for modern transport systems.

Deployment Diagram



Deployment Diagram

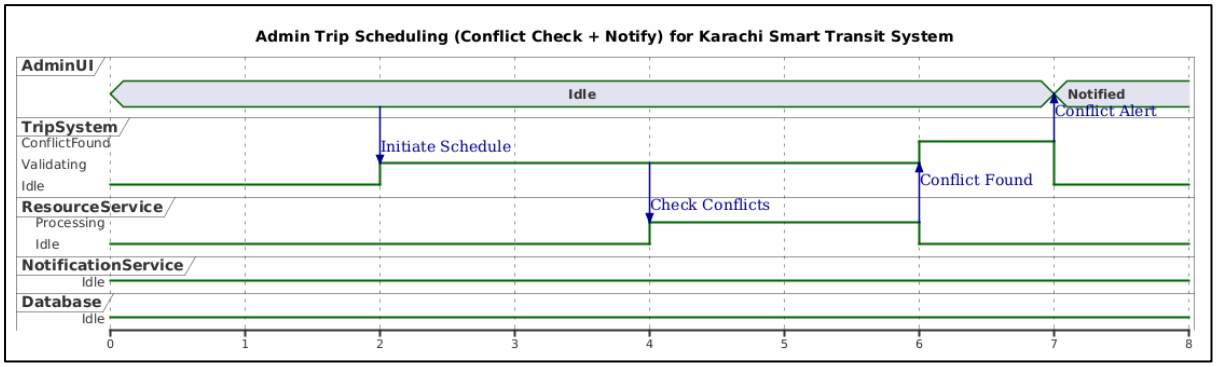
The Deployment Diagram represents the physical architecture of the system by showing how software components are distributed across hardware nodes. It follows a three-tier architecture consisting of the Client Tier, Application Server, and Database Server.

Clients (web browsers) interact with the Flask-based application server via HTTP and WebSocket protocols. The application server processes requests, executes business logic, and communicates with the MySQL database for data storage and retrieval.

This diagram helps in understanding system scalability, network communication, and deployment strategy.

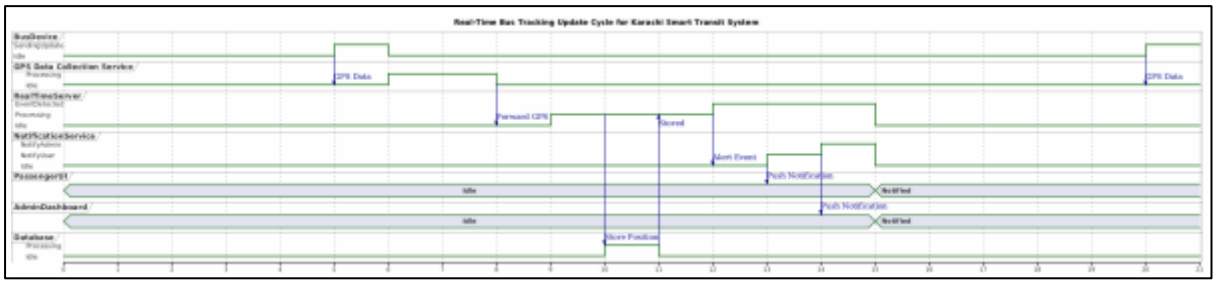
Timing Diagram

Admin Trip Scheduling



Passenger Trip Booking Flow

Real-Time Bus Tracking Update Cycle



Timing Diagram

The Timing Diagram focuses on the behavior of system components over time, showing how different processes are synchronized. It captures events such as admin trip scheduling, passenger booking flow, and real-time tracking updates.

The diagram illustrates how state changes occur in response to events and how timing constraints affect system operations. For example, GPS updates occur at regular intervals, and booking processes follow a defined sequence.

This is particularly useful for analyzing performance, synchronization, and time-dependent behavior.

Architectural View (4+1 Model)

Logical View

The Logical View describes the system's internal structure in terms of classes and their relationships. It focuses on the design of core components such as User, Trip, Booking, and Payment, and explains how they interact logically.

This view ensures that the system is well-structured, modular, and easy to maintain by clearly defining responsibilities and relationships between components.

Process View

The Process View focuses on the runtime behavior of the system, including concurrency, synchronization, and communication between processes. It highlights how multiple users can perform operations such as booking and tracking simultaneously without conflicts.

Critical sections like seat allocation and payment processing are managed carefully to ensure data consistency and reliability.

Development View

The Development View explains how the system is organized from a developer's perspective. It includes the project structure, modules, and technologies used such as Flask for backend, React for frontend, and MySQL for database management.

It also describes different layers like presentation, application, domain, and data layers, ensuring a clean and maintainable architecture.

Physical View

The Physical View represents how the system is deployed in a real-world environment. It includes hardware components, network configuration, and communication protocols.

The system uses a 3-tier architecture where clients communicate with the server, and the server interacts with the database. It also supports scalability through load balancing and secure communication using HTTPS.

+1 (Use Case View)

The Use Case View captures the system's functionality from the end-user perspective. It summarizes all key interactions between actors and the system, including booking, payment, scheduling, and reporting.

This view acts as a bridge between requirements and design, ensuring that all user needs are properly addressed in the system.